

基于邻接多重表的小说人物关联分析系统开发文档

1. 项目概述

1.1 项目名称

基于邻接多重表与 Jaccard 系数的小说人物关联分析系统。

1.2 项目背景

小说中包含大量人物及人物关系。人物之间是否在同一章节出现，可以作为关系存在的依据；共同出现章节所占比例，可以用于衡量人物关联程度。

本系统以小说章节为最小统计单位。用户每次从文本文件中导入一章小说，程序从该章节中提取人物名称，经过别名统一和人工确认后，将本章人物加入累计统计。

系统使用无向加权图描述人物关系：

- 顶点表示小说人物。
- 无向边表示两个人物至少在一个章节中共同出现。
- 边权值表示两个人物的 Jaccard 关联度。
- 邻接多重表用于存储无向人物关系网。
- DFS 和 BFS 用于遍历人物关系网。
- 孩子链表用于存储 DFS 或 BFS 生成的遍历树。

1.3 设计目标

系统应达到以下目标：

1. 掌握无向网邻接多重表的构造方法。
2. 实现邻接多重表中边结点的插入、查找和删除。
3. 实现基于邻接多重表的 DFS 和 BFS。
4. 使用孩子链表存储遍历生成树。
5. 支持逐章导入小说并累计统计。
6. 使用 Jaccard 系数计算人物关联程度。
7. 使用图形界面展示人物、人物关系和遍历结果。
8. 尽量从文件读取初始数据和小说章节。
9. 项目保存数据必须采用二进制格式。
10. 遵循低耦合、高内聚的设计原则。
11. 不使用全局业务变量。
12. 不使用固定长度的传统数组存储业务数据。
13. 使用标准容器和动态内存管理。
14. 每次增、删、改操作后执行数据一致性验证。

2. 系统适用范围

本系统适用于以下场景：

- 数据结构课程设计。
- 小说人物共现关系分析。
- 人物关系图可视化。
- 人物关系群体和间接关系分析。
- 小说章节人物统计。
- DFS、BFS 和一般树存储结构演示。

系统主要面向中小规模小说。建议单个项目满足以下规模：

- 人物数量不超过 5000。
- 章节数量不超过 10000。
- 单章识别人物数量通常不超过 200。
- 人物关系边数量不超过 100000。

对于人物极多、章节极长或需要复杂自然语言理解的项目，应增加数据库和专业自然语言处理模块。

3. 术语定义

3.1 人物出现章节数

人物 A 的出现章节数记为：

$$C(A)$$

同一人物在同一章节中出现多次，章节计数仍只增加一次。

3.2 人物共同出现章节数

人物 A 和人物 B 的共同出现章节数记为：

$$C(A, B)$$

只要两个人物在同一章节中都出现，该章节就计入一次共同出现。

3.3 Jaccard 关联度

人物 A 和人物 B 的 Jaccard 关联度定义为：

$$J(A, B) = \frac{C(A, B)}{C(A) + C(B) - C(A, B)}$$

Jaccard 关联度取值范围为：

$$0 < J(A, B) \leq 1$$

当两个人物从未共同出现时，不建立人物关系边。

3.4 标准人物名

标准人物名是系统内部统一使用的人物名称，例如“孙悟空”。

3.5 人物别名

人物别名是指向标准人物名的其他称呼，例如：

悟空	-> 孙悟空
行者	-> 孙悟空
齐天大圣	-> 孙悟空

4. 总体需求

4.1 功能需求

系统应提供以下功能。

4.1.1 项目管理

- 新建人物分析项目。
- 打开二进制项目文件。
- 保存当前项目。
- 另存为新的二进制项目文件。
- 查看项目基本信息。
- 自动保存可作为可选功能。

4.1.2 初始数据读取

新建项目时，可以从文本文件读取：

- 人物标准名称词典。
- 人物别名词典。
- 系统配置。
- 关系显示阈值。
- 文本编码设置。

初始数据文本文件统一使用 UTF-8 编码。

4.1.3 章节导入

- 一次导入一个章节文本文件。
- 从文件中读取章节编号、标题和正文。
- 自动提取人物名称。
- 对提取结果进行别名统一。
- 对同一章节中的人物进行去重。
- 在确认导入前允许人工增删人物。
- 检查章节编号是否重复。
- 保存章节正文和确认后的人物集合。
- 导入成功后重新计算统计结果。

4.1.4 章节管理

- 查看已经导入的章节。
- 查看某章正文。
- 查看某章识别出的人物。
- 修改章节标题。
- 修改章节人物集合。
- 删除章节。
- 重新从章节正文提取人物。
- 根据所有章节重新构建人物关系图。

4.1.5 人物管理

- 查看所有人物。
- 查询人物。
- 查看人物出现章节数。
- 查看人物出现的具体章节。
- 新增标准人物名。
- 修改人物名称。
- 添加人物别名。
- 删除人物别名。
- 合并重复人物。
- 删除没有出现在任何章节中的人物。

4.1.6 关系管理

- 查看全部人物关系。
- 查看共同出现章节数。
- 查看 Jaccard 关联度。
- 查询两个人物之间的关系。
- 按关联度排序。
- 按共同出现章节数排序。
- 设置关系显示阈值。
- 查看两个人物共同出现的章节。

邻接多重表的边插入和删除操作必须在底层数据结构中实现。

章节导入时：

- 若关系边不存在，插入新边。
- 若关系边已经存在，增加共同章节数。

章节删除时：

- 减少相应共同章节数。
- 共同章节数降为零时删除关系边。

为了保证统计结果完全一致，正式版本推荐在章节发生修改或删除后，根据所有章节记录重新构建人物图。

4.1.7 图遍历

- 选择一个人物作为遍历起点。
- 执行深度优先遍历。
- 执行广度优先遍历。
- 显示遍历序列。
- 生成遍历树。
- 使用孩子链表存储遍历树。
- 图形化显示遍历树。
- 支持非连通图的完整遍历。

4.1.8 图形化显示

- 人物显示为图结点。
- 人物关系显示为无向边。
- 结点大小可根据人物出现章节数调整。
- 边宽可根据 Jaccard 关联度调整。
- 支持缩放、拖动和平移。
- 支持搜索并定位人物。
- 支持设置最低关联度阈值。
- 支持点击人物查看详情。
- 支持点击关系边查看详情。
- 支持高亮选中人物的一阶关系。
- 支持显示 DFS 或 BFS 遍历动画。

4.1.9 数据验证

每次章节导入、修改、删除及项目加载后，应验证：

- 顶点编号是否合法。
- 边的两个端点是否存在。
- 边是否同时连接到两个端点的边链。
- 是否存在重复边。

- 共同章节数是否为正数。
- 共同章节数是否超过任一人物出现章节数。
- Jaccard 值是否正确。
- 章节中的人物编号是否重复。
- 人物出现章节数是否与章节记录一致。
- 别名是否指向有效标准人物。
- 二进制文件是否完整。

5. 非功能需求

5.1 可维护性

- 每个类只承担相对单一的职责。
- 界面层不直接操作邻接多重表。
- 文件读取模块不负责统计计算。
- 图算法模块不负责界面绘制。
- 二进制序列化模块不依赖具体窗口。
- 业务对象不直接弹出对话框。
- 错误通过返回值、异常或结果对象向上层传递。

5.2 安全性

- 不信任外部文本文件内容。
- 对文件长度、字段长度和记录数量设置上限。
- 保存时使用临时文件和原子替换。
- 加载二进制文件时检查魔数、版本、长度和摘要。
- 禁止直接将内存结构按字节写入文件。
- 禁止保存裸指针地址。

5.3 性能

- 人物名称查找应接近常数时间。
- 人物关系查找应接近常数时间。
- 图遍历复杂度为 $O(V + E)$ 。
- 图重建复杂度主要为：

$$O\left(\sum_{c=1}^N k_c^2\right)$$

其中 k_c 表示第 c 章中的人物数量。

5.4 易用性

- 导入章节前显示提取结果。
- 所有可能导致数据丢失的操作需要确认。
- 错误提示应包含文件名、章节编号和原因。

- 界面应提供状态栏和操作日志。
- 长时间操作应显示进度。
- 导入失败时不得修改当前项目数据。

6. 技术选型

6.1 开发语言

C++17。

6.2 图形界面

Qt 6 Widgets。

主要使用：

- QMainWindow
- QSplitter
- QDockWidget
- QGraphicsView
- QGraphicsScene
- QTreeView
- QTableView
- QListView
- QTextEdit
- QFileDialog
- QMessageBox
- QProgressDialog

6.3 构建系统

CMake 3.21 或更高版本。

6.4 测试框架

优先使用 Qt Test。

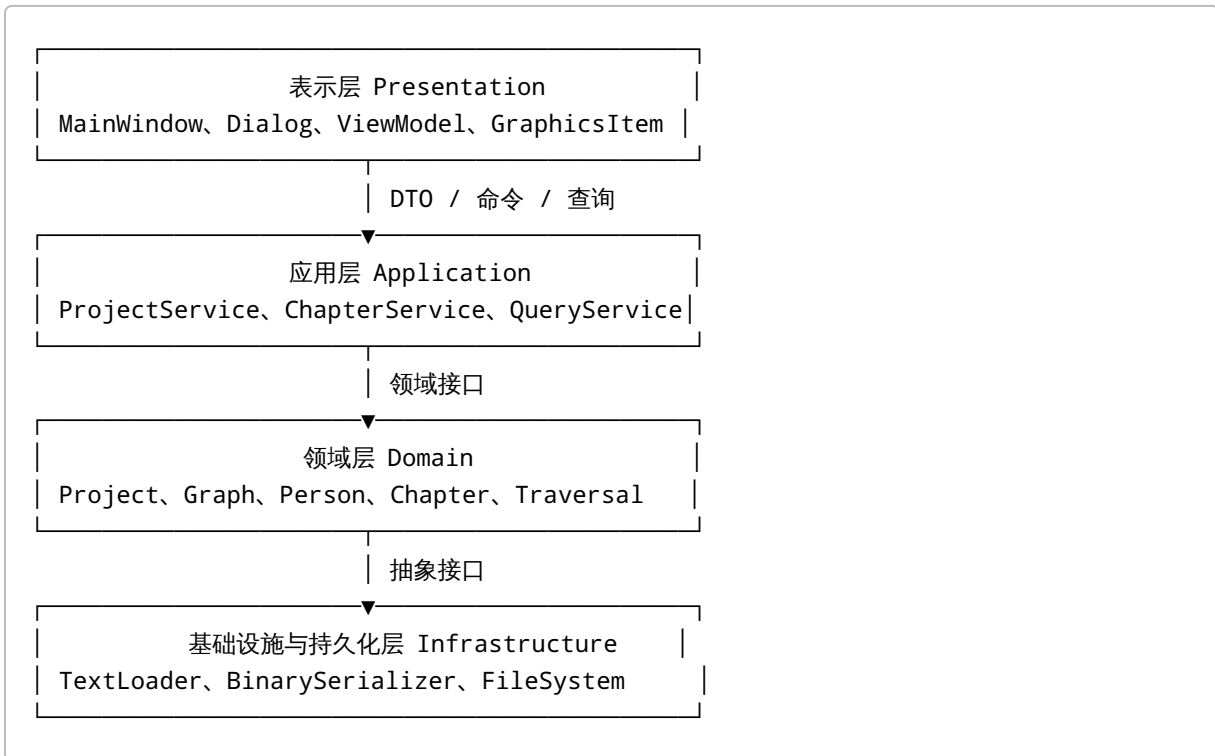
核心领域层保持纯 C++ 时，也可以使用 Catch2 或 GoogleTest。

6.5 字符编码

- 文本文件：UTF-8。
- 程序内部文本：UTF-8 `std::string`。
- Qt 界面边界处进行 `QString` 与 UTF-8 的转换。

7. 总体架构

系统采用分层架构：



7.1 表示层职责

- 接收用户操作。
- 展示人物、章节、关系和遍历结果。
- 收集输入参数。
- 调用应用层接口。
- 展示错误信息。

表示层不得直接：

- 修改人物出现次数。
- 创建邻接多重表边结点。
- 计算 Jaccard 系数。
- 解析二进制文件。
- 维护人物别名映射。

7.2 应用层职责

- 组织完整用例。
- 控制导入章节流程。
- 控制保存和加载流程。

- 调用领域对象完成业务操作。
- 管理事务式更新。
- 将领域对象转换为界面 DTO。

7.3 领域层职责

- 管理人物、章节和人物关系。
- 实现邻接多重表。
- 实现 Jaccard 计算。
- 实现 DFS 和 BFS。
- 实现孩子链表。
- 执行领域一致性验证。

7.4 基础设施层职责

- 读取 UTF-8 文本文件。
- 保存和加载二进制项目文件。
- 计算文件摘要。
- 执行文件原子替换。
- 提供日志输出。

8. 全局变量与数组使用约束

8.1 禁止使用的形式

禁止定义以下业务全局变量：

```
Person persons[1000];
Edge edges[10000];
int personCount;
Graph globalGraph;
Project* currentProject;
```

禁止依赖固定长度字符数组存储名称：

```
char name[20];
```

禁止使用全局访问器隐藏全局状态：

```
Project& getGlobalProject();
```

8.2 推荐形式

所有项目状态由 `Project` 对象持有：

```
int main(int argc, char* argv[])
{
    QApplication application(argc, argv);

    auto serializer = std::make_unique<BinaryProjectSerializer>();
    auto extractor = std::make_unique<DictionaryNameExtractor>();

    ProjectApplicationService service(
        std::move(serializer),
        std::move(extractor)
    );

    MainWindow window(service);
    window.show();

    return application.exec();
}
```

`ProjectApplicationService` 由 `MainWindow` 引用，不通过全局变量访问。

业务集合使用标准容器：

```
std::vector<PersonId>
std::unordered_map<PersonId, std::unique_ptr<PersonVertex>>
std::unordered_map<EdgeKey, std::unique_ptr<EdgeNode>, EdgeKeyHash>
std::vector<ChapterRecord>
std::unordered_map<std::string, PersonId>
```

传统数组仅允许用于很小的局部缓冲区或标准算法内部实现，并且应优先使用 `std::array`。

9. 核心领域模型

9.1 类型定义

```
using PersonId = std::uint32_t;
using EdgeId = std::uint64_t;
using ChapterId = std::uint64_t;
```

所有编号均由项目对象内部生成，不由界面直接分配。

9.2 人物顶点

```
struct PersonVertex
{
    PersonId id {};
    std::string canonicalName;
    std::uint32_t chapterCount {};
    EdgeNode* firstEdge {};
};
```

说明：

- `canonicalName` 为标准人物名。
- `chapterCount` 为角色出现章节数。
- `firstEdge` 指向邻接多重表中的第一条关联边。
- `firstEdge` 为非拥有指针。
- 人物对象的生命周期由图对象统一管理。

9.3 边结点

```
struct EdgeNode
{
    EdgeId id {};

    PersonId endpointA {};
    PersonId endpointB {};

    std::uint32_t coChapterCount {};
    double jaccard {};

    EdgeNode* linkA {};
    EdgeNode* linkB {};
};
```

说明：

- 一条无向边仅创建一个 `EdgeNode`。
- `linkA` 指向端点 A 边链中的下一条边。
- `linkB` 指向端点 B 边链中的下一条边。
- 边对象由图对象统一拥有。
- 所有链指针均为非拥有指针。
- 删除边时先从两个端点的边链中摘除，再释放边对象。

9.4 无序人物对

```
struct EdgeKey
{
    PersonId low {};
    PersonId high {};

    static EdgeKey make(PersonId a, PersonId b)
    {
        return a < b ? EdgeKey{a, b} : EdgeKey{b, a};
    }

    bool operator==(const EdgeKey&) const = default;
};
```

使用规范化的 `EdgeKey` 防止出现重复边。

9.5 章节记录

```
struct ChapterRecord
{
    ChapterId id {};
    std::string chapterKey;
    std::string title;
    std::string sourceFileName;
    std::string contentUtf8;
    std::vector<PersonId> persons;
};
```

约束：

- `chapterKey` 在项目中唯一。
- `persons` 中人物编号不得重复。
- 保存项目时，章节正文存入二进制项目文件。
- 原始文本文件只作为导入来源。

9.6 人物别名表

```
class AliasDictionary
{
public:
    void addAlias(std::string alias, PersonId target);
    void removeAlias(const std::string& alias);
};
```

```

        std::optional<PersonId> resolve(const std::string& name) const;

private:
    std::unordered_map<std::string, PersonId> aliasToPerson_;
};

```

别名表只负责名称解析，不负责创建人物或修改章节。

9.7 项目聚合对象

```

class NovelRelationProject
{
public:
    ProjectMetadata& metadata();
    const ProjectMetadata& metadata() const;

    AdjacencyMultilistGraph& graph();
    const AdjacencyMultilistGraph& graph() const;

    ChapterCollection& chapters();
    const ChapterCollection& chapters() const;

    AliasDictionary& aliases();
    const AliasDictionary& aliases() const;

    void clear();

private:
    ProjectMetadata metadata_;
    AdjacencyMultilistGraph graph_;
    ChapterCollection chapters_;
    AliasDictionary aliases_;
};

```

项目对象是整个业务状态的拥有者。

10. 邻接多重表设计

10.1 图类定义

```

class AdjacencyMultilistGraph
{
public:

```

```

PersonId addPerson(const std::string& canonicalName);
bool removePerson(PersonId id);
bool renamePerson(PersonId id, const std::string& newName);

EdgeNode& insertEdge(
    PersonId a,
    PersonId b,
    std::uint32_t coChapterCount
);

bool deleteEdge(PersonId a, PersonId b);

EdgeNode* findEdge(PersonId a, PersonId b);
const EdgeNode* findEdge(PersonId a, PersonId b) const;

PersonVertex* findPerson(PersonId id);
const PersonVertex* findPerson(PersonId id) const;

std::vector<PersonId> neighbors(PersonId id) const;

void clearStatistics();
void recomputeAllJaccard();
ValidationReport validate() const;

private:
    EdgeNode* nextEdge(
        const EdgeNode* edge,
        PersonId currentVertex
    ) const;

    PersonId opposite(
        const EdgeNode& edge,
        PersonId currentVertex
    ) const;

private:
    PersonId nextPersonId_ {1};
    EdgeId nextEdgeId_ {1};

    std::unordered_map<
        PersonId,
        std::unique_ptr<PersonVertex>
    > persons_;

    std::unordered_map<
        std::string,
        PersonId
    > personNameIndex_;

```

```

std::unordered_map<
    EdgeKey,
    std::unique_ptr<EdgeNode>,
    EdgeKeyHash
> edges_;
};

```

10.2 内存所有权

- `persons_` 拥有人物对象。
- `edges_` 拥有边对象。
- `firstEdge`、`linkA`、`linkB` 均不拥有对象。
- 禁止对这些裸指针执行 `delete`。
- 图对象销毁时，智能指针自动释放人物和边。

虽然 `unordered_map` 可能发生重新散列，但 `unique_ptr` 所指对象的实际地址保持不变，因此边链中的指针仍然有效。

10.3 插入边

插入边步骤：

1. 检查两个端点是否存在。
2. 检查两个端点是否相同。
3. 构造规范化 `EdgeKey`。
4. 检查边是否已经存在。
5. 创建 `EdgeNode`。
6. 将新边插入端点 A 的边链头部。
7. 将新边插入端点 B 的边链头部。
8. 将边对象放入 `edges_`。
9. 计算 Jaccard 值。
10. 执行局部一致性检查。

伪代码：

```

EdgeNode& AdjacencyMultilistGraph::insertEdge(
    PersonId a,
    PersonId b,
    std::uint32_t coChapterCount)
{
    if (a == b) {
        throw DomainError("不允许创建人物自环");
    }

    auto* vertexA = findPerson(a);

```

```

    auto* vertexB = findPerson(b);

    if (vertexA == nullptr || vertexB == nullptr) {
        throw DomainError("人物端点不存在");
    }

    const EdgeKey key = EdgeKey::make(a, b);

    if (auto iterator = edges_.find(key); iterator != edges_.end()) {
        return *iterator->second;
    }

    auto edge = std::make_unique<EdgeNode>();
    edge->id = nextEdgeId_++;
    edge->endpointA = a;
    edge->endpointB = b;
    edge->coChapterCount = coChapterCount;

    edge->linkA = vertexA->firstEdge;
    edge->linkB = vertexB->firstEdge;

    EdgeNode* rawEdge = edge.get();

    vertexA->firstEdge = rawEdge;
    vertexB->firstEdge = rawEdge;

    edges_.emplace(key, std::move(edge));

    return *rawEdge;
}

```

10.4 删除边

删除边时，需要分别定位该边在两个端点边链中的前驱。

应将“从某个顶点边链中摘除边”的过程封装为私有函数：

```
void unlinkFromVertex(PersonVertex& vertex, EdgeNode& target);
```

删除流程：

1. 查找目标边。
2. 从端点 A 的边链摘除。
3. 从端点 B 的边链摘除。
4. 从 `edges_` 中删除边对象。

5. 验证两个端点的边链。

10.5 遍历某个人物的所有边

```
EdgeNode* edge = person.firstEdge;

while (edge != nullptr) {
    PersonId neighbor = opposite(*edge, person.id);

    edge = edge->endpointA == person.id
        ? edge->linkA
        : edge->linkB;
}
```

所有图算法都应通过统一的边访问接口进行遍历，避免在多个模块重复编写端点判断逻辑。

11. 章节文本格式

每个章节使用独立 UTF-8 文本文件。

推荐格式：

```
@chapter=001
@title=第一章 初入江湖

张三走进客栈，看见李四正在与王五交谈。
张三向李四询问了城中的情况.....
```

规则：

- `@chapter=` 表示章节唯一编号。
- `@title=` 表示章节标题。
- 空行后为正文。
- 元数据行必须位于文件开头。
- 缺少章节编号时，导入界面必须要求用户补充。
- 文件大小默认不得超过 20 MB。
- 非 UTF-8 文件应提示用户转换编码。

解析结果：

```
struct ParsedChapterText
{
    std::string chapterKey;
```

```
std::string title;  
std::string contentUtf8;  
};
```

12. 初始人物词典格式

12.1 标准人物文件

文件名示例：

```
persons.txt
```

内容格式为一行一个标准人物名：

```
张三  
李四  
王五  
孙悟空  
唐僧  
猪八戒  
沙僧
```

规则：

- 忽略空行。
- 以 `#` 开头的行为注释。
- 名称两端空白应被移除。
- 不允许重复名称。

12.2 别名文件

文件名示例：

```
aliases.txt
```

使用制表符分隔：

```
悟空 孙悟空  
行者 孙悟空  
大圣 孙悟空
```

齐天大圣 孙悟空
三藏 唐僧
八戒 猪八戒

左侧为别名，右侧为标准人物名。

加载时应验证：

- 标准人物名已存在。
- 一个别名只能映射到一个标准人物。
- 别名不能与其他标准人物名冲突。
- 不允许产生循环映射。

13. 人物提取模块

13.1 提取接口

```
class INameExtractor
{
public:
    virtual ~INameExtractor() = default;

    virtual std::vector<std::string> extract(
        const std::string& contentUtf8
    ) const = 0;
};
```

界面层和章节服务只依赖 `INameExtractor`，不依赖具体实现。

13.2 基于词典的提取器

```
class DictionaryNameExtractor final : public INameExtractor
{
public:
    explicit DictionaryNameExtractor(
        std::shared_ptr<const NameDictionary> dictionary
    );

    std::vector<std::string> extract(
        const std::string& contentUtf8
    ) const override;
```

```
private:
    std::shared_ptr<const NameDictionary> dictionary_;
};
```

课程设计版本可以使用词典匹配。

提取流程：

1. 对标准人物名和别名按长度降序排列。
2. 在正文中搜索人物名称。
3. 记录匹配到的名称。
4. 将别名转换为标准人物编号。
5. 对人物编号去重。
6. 返回本章人物集合。
7. 在界面中允许人工确认。

人物名称应采用“较长名称优先”的原则，减少“齐天大圣”被拆分为“大圣”的情况。

13.3 人工确认

自动提取完成后，界面显示：

- 已识别人物。
- 匹配到的别名。
- 未确认名称。
- 本章人物数量。

用户可以：

- 删除误识别人物。
- 添加漏识别人物。
- 将新名称设为标准人物。
- 将新名称设为已有角色的别名。

人工确认后的集合才进入统计。

14. 统计构建策略

14.1 数据源原则

章节记录是统计数据的权威来源。

以下数据均属于派生数据：

- 人物出现章节数。
- 人物共同出现章节数。
- Jaccard 关联度。
- 邻接多重表中的人物关系边。

当章节发生新增、修改或删除后，可以根据全部章节重新生成派生数据。

这种策略可以降低增量修改逻辑的复杂度，并减少统计错误。

14.2 重建算法

```
void GraphStatisticsBuilder::rebuild(
    NovelRelationProject& project)
{
    auto& graph = project.graph();

    graph.clearStatistics();

    for (const ChapterRecord& chapter : project.chapters().all()) {
        const auto uniquePersons = deduplicate(chapter.persons);

        for (PersonId personId : uniquePersons) {
            graph.findPerson(personId)->chapterCount++;
        }

        for (std::size_t i = 0; i < uniquePersons.size(); ++i) {
            for (std::size_t j = i + 1;
                j < uniquePersons.size();
                ++j) {
                PersonId a = uniquePersons[i];
                PersonId b = uniquePersons[j];

                EdgeNode* edge = graph.findEdge(a, b);

                if (edge == nullptr) {
                    edge = &graph.insertEdge(a, b, 0);
                }

                edge->coChapterCount++;
            }
        }
    }
}
```

```
graph.recomputeAllJaccard();  
}
```

14.3 Jaccard 计算

```
double calculateJaccard(  
    std::uint32_t countA,  
    std::uint32_t countB,  
    std::uint32_t coCount)  
{  
    const std::uint32_t denominator =  
        countA + countB - coCount;  
  
    if (denominator == 0) {  
        return 0.0;  
    }  
  
    return static_cast<double>(coCount)  
        / static_cast<double>(denominator);  
}
```

14.4 修改事务

章节新增、修改和删除采用临时副本方式：

1. 复制章节集合或创建临时项目数据。
2. 在临时数据上执行操作。
3. 重建人物图。
4. 执行完整验证。
5. 验证成功后替换当前项目状态。
6. 验证失败时丢弃临时数据。

可以使用：

```
NovelRelationProject candidate = currentProject.clone();
```

若项目较大，可将章节集合和图分别构建到临时对象中，再通过 `swap` 提交。

15. DFS 与 BFS

15.1 邻接访问接口

图遍历模块不直接读取图内部容器，统一通过只读接口访问：

```
class IGraphView
{
public:
    virtual ~IGraphView() = default;

    virtual std::vector<PersonId> vertices() const = 0;
    virtual std::vector<PersonId> neighbors(PersonId id) const = 0;
};
```

`AdjacencyMultilistGraph` 可以实现该接口。

15.2 深度优先遍历

推荐使用显式栈，避免递归层数过深。

```
TraversalResult depthFirstSearch(
    const IGraphView& graph,
    PersonId start);
```

结果包含：

```
struct TraversalResult
{
    std::vector<PersonId> order;
    ChildListTree tree;
};
```

15.3 广度优先遍历

使用 `std::queue<PersonId>`：

```
TraversalResult breadthFirstSearch(
    const IGraphView& graph,
    PersonId start);
```

访问一个未访问邻居时：

- 将邻居标记为已访问。
- 将邻居加入队列。
- 在遍历树中将邻居添加为当前人物的孩子。

15.4 非连通图

若要求遍历所有人物：

1. 先从用户指定人物开始遍历。
2. 按人物编号或名称顺序查找未访问顶点。
3. 对每个未访问顶点重新开始一次遍历。
4. 创建虚拟根结点“全部人物”。
5. 每个连通分量的起始人物作为虚拟根的孩子。

16. 孩子链表设计

16.1 孩子结点

```
struct ChildLink
{
    PersonId child {};
    std::unique_ptr<ChildLink> next;
};
```

16.2 树结点

```
struct TreeNode
{
    PersonId person {};
    std::unique_ptr<ChildLink> firstChild;
};
```

16.3 遍历树

```
class ChildListTree
{
public:
    void createNode(PersonId person);
    void addChild(PersonId parent, PersonId child);
};
```



```

    const TreeNode* findNode(PersonId person) const;

    std::vector<PersonId> children(PersonId parent) const;

private:
    std::unordered_map<
        PersonId,
        std::unique_ptr<TreeNode>
    > nodes_;
};

```

`addChild` 只负责维护孩子链表，不负责图遍历。

17. 应用服务设计

17.1 项目应用服务

```

class ProjectApplicationService
{
public:
    explicit ProjectApplicationService(
        std::unique_ptr<IProjectSerializer> serializer,
        std::unique_ptr<INameExtractor> extractor
    );

    Result<void> newProject(
        const NewProjectOptions& options
    );

    Result<void> openProject(
        const std::filesystem::path& path
    );

    Result<void> saveProject();

    Result<void> saveProjectAs(
        const std::filesystem::path& path
    );

    Result<ChapterPreview> previewChapter(
        const std::filesystem::path& textFile
    );
};

```

```

Result<void> confirmChapterImport(
    const ConfirmChapterCommand& command
);

Result<void> modifyChapter(
    const ModifyChapterCommand& command
);

Result<void> deleteChapter(ChapterId id);

Result<GraphSnapshot> graphSnapshot(
    double minimumJaccard
) const;

Result<TraversalResultDto> runDfs(PersonId start) const;
Result<TraversalResultDto> runBfs(PersonId start) const;

private:
    NovelRelationProject project_;
    std::filesystem::path currentFile_;

    std::unique_ptr<IProjectSerializer> serializer_;
    std::unique_ptr<INameExtractor> extractor_;
};

```

17.2 Result 类型

不建议在领域层中弹出错误对话框。

```

template<typename T>
class Result
{
public:
    static Result success(T value);
    static Result failure(Error error);

    bool hasValue() const;
    const T& value() const;
    const Error& error() const;
};

```

界面层收到失败结果后，再通过 `QMessageBox` 展示错误。

18. 界面设计

18.1 主窗口布局

主窗口使用 `QMainWindow` 和 `QSplitter`。

菜单栏：文件 章节 人物 关系 遍历 工具 帮助				
章节管理区	人物关系图		人物/关系详情区	
已导入章节	QGraphicsView	人物出现次数		
章节搜索		关联人物列表		
导入章节		Jaccard值		
修改/删除		共同章节列表		
遍历控制：起点 DFS BFS 播放 停止 显示遍历树				
状态栏：项目路径、人物数、边数、章节数、保存状态				

18.2 章节导入对话框

包含：

- 文件路径。
- 章节编号。
- 章节标题。
- 正文预览。
- 自动识别人物列表。
- 搜索人物输入框。
- 添加人物按钮。
- 删除人物按钮。
- 添加别名按钮。
- 确认导入按钮。
- 取消按钮。

18.3 人物关系图

建议定义：

```
class PersonNodeItem : public QGraphicsObject;
class RelationEdgeItem : public QGraphicsItem;
class RelationGraphSceneController;
```

`PersonNodeItem` 只负责显示和交互。

`RelationGraphSceneController` 接收 `GraphSnapshot`，不持有领域图的可修改引用。

18.3.1 GraphSnapshot

```
struct PersonNodeDto
{
    PersonId id {};
    std::string name;
    std::uint32_t chapterCount {};
};

struct RelationEdgeDto
{
    EdgeId id {};
    PersonId a {};
    PersonId b {};
    std::uint32_t coChapterCount {};
    double jaccard {};
};

struct GraphSnapshot
{
    std::vector<PersonNodeDto> nodes;
    std::vector<RelationEdgeDto> edges;
};
```

18.3.2 显示规则

- 结点直径根据人物出现章节数缩放。
- 边宽根据 Jaccard 值缩放。
- 边标签默认隐藏。
- 鼠标悬停时显示边权值。
- 双击人物打开详情。
- 选中人物时高亮全部相邻边。
- 低于阈值的边不进入当前快照。

18.3.3 布局算法

第一版可使用圆形布局：

$$x_i = x_0 + r \cos \frac{2\pi i}{n}$$
$$y_i = y_0 + r \sin \frac{2\pi i}{n}$$

后续可增加简单力导向布局。

图形布局属于表示层，不修改领域数据。

19. 二进制项目文件设计

19.1 文件扩展名

建议使用：

```
.nprg
```

含义为 Novel Person Relation Graph。

19.2 保存原则

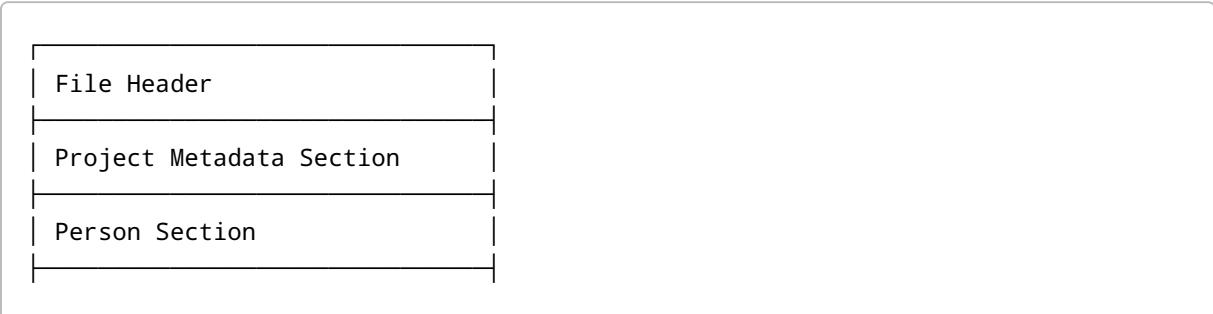
程序生成的完整项目数据必须保存为二进制文件。

禁止使用 JSON、XML、CSV 或普通文本作为项目保存格式。

文本格式只允许用于：

- 初始人物词典。
- 初始别名词典。
- 章节导入文件。
- 可选配置文件。
- 日志文件。

19.3 文件结构



Alias Section
Chapter Section
Graph Statistics Section
File Digest

19.3.1 文件头

字段	类型	说明
Magic	8 字节	固定为 NPRGRAPH
FormatVersion	uint32	文件格式版本
EndianMarker	uint32	固定为 0x01020304
PayloadLength	uint64	数据区长度
SectionCount	uint32	区段数量
Reserved	uint32	保留字段

19.3.2 字符串格式

字符串采用：

uint32 字节长度
UTF-8 字节内容

必须检查：

- 字符串长度不超过预设上限。
- 剩余文件长度足够。
- UTF-8 内容合法或可安全替换非法字符。

19.3.3 区段格式

每个区段使用：

字段	类型
SectionType	uint32
SectionVersion	uint32

字段	类型
SectionLength	uint64
SectionPayload	变长

未知区段可以根据 `SectionLength` 跳过，便于未来扩展。

19.3.4 人物区段

每个人物保存：

- `PersonId`
- 标准人物名
- 人物出现章节数

`firstEdge` 不保存，因为指针地址在重新启动后无效。

19.3.5 关系区段

每条边保存：

- `EdgeId`
- `endpointA`
- `endpointB`
- `coChapterCount`
- `jaccard`

`linkA` 和 `linkB` 不保存。

加载时根据端点重新建立邻接多重表链。

19.3.6 章节区段

每章保存：

- `ChapterId`
- `chapterKey`
- 标题
- 原始文件名
- 正文 UTF-8 内容
- 人物数量
- 人物编号列表

19.3.7 别名区段

每条别名记录保存：

- 别名字符串
- 目标 `PersonId`

19.3.8 文件摘要

文件末尾保存整个数据区的 SHA-256 摘要。

加载时重新计算摘要并比较。

19.4 序列化接口

```
class IProjectSerializer
{
public:
    virtual ~IProjectSerializer() = default;

    virtual Result<void> save(
        const NovelRelationProject& project,
        const std::filesystem::path& path
    ) const = 0;

    virtual Result<NovelRelationProject> load(
        const std::filesystem::path& path
    ) const = 0;
};
```

具体实现：

```
class BinaryProjectSerializer final
    : public IProjectSerializer
{
public:
    Result<void> save(
        const NovelRelationProject& project,
        const std::filesystem::path& path
    ) const override;

    Result<NovelRelationProject> load(
        const std::filesystem::path& path
```



```
    ) const override;  
};
```

19.5 原子保存

保存流程：

1. 在目标文件同目录创建临时文件。
2. 将完整数据写入临时文件。
3. 刷新输出缓冲区。
4. 关闭文件。
5. 重新打开并执行必要验证。
6. 将旧项目文件重命名为备份。
7. 将临时文件原子替换为目标文件。
8. 删除旧备份。

若保存失败，原项目文件应保持可用。

20. 加载流程

打开项目文件时：

1. 读取并验证魔数。
2. 检查文件格式版本。
3. 检查端序标记。
4. 检查数据区长度。
5. 读取各区段。
6. 创建人物对象。
7. 创建章节对象。
8. 创建别名映射。
9. 根据边记录插入邻接多重表。
10. 验证文件摘要。
11. 执行项目完整验证。
12. 可选地根据章节重新构建统计。
13. 比较保存的统计与重建统计。
14. 所有步骤成功后替换当前项目。

加载过程中不得直接覆盖当前已打开项目。

21. 一致性验证设计

21.1 验证结果

```
enum class ValidationSeverity
{
    Information,
    Warning,
    Error
};

struct ValidationIssue
{
    ValidationSeverity severity;
    std::string code;
    std::string message;
};

struct ValidationReport
{
    std::vector<ValidationIssue> issues;

    bool isValid() const;
};
```

21.2 图结构验证

至少检查：

1. 每个顶点的 `firstEdge` 是否为空或指向有效边。
2. 每条边的两个端点是否存在。
3. 每条边是否在端点 A 的边链中出现一次。
4. 每条边是否在端点 B 的边链中出现一次。
5. 每条边是否在其他无关顶点边链中出现。
6. 是否存在边链环。
7. 是否存在重复 `EdgeKey`。
8. 是否存在人物自环。
9. `coChapterCount` 是否大于零。
10. Jaccard 是否位于合法范围。
11. Jaccard 是否与公式结果一致。

浮点比较使用误差：

```
constexpr double epsilon = 1e-9;
```

此常量应定义为类内 `static constexpr` 或函数局部常量，不定义可修改的全局变量。

21.3 统计验证

通过章节重新计算：

- 每个人物出现章节数。
- 每个人物对共同章节数。
- 每条边 Jaccard 值。

将重新计算结果与当前图比较。

21.4 章节验证

- 章节编号唯一。
- 章节 ID 唯一。
- 人物编号存在。
- 人物编号不重复。
- 正文大小合法。
- 标题和章节编号长度合法。

22. 异常与错误处理

22.1 错误分类

```
enum class ErrorCode
{
    FileNotFound,
    FileOpenFailed,
    InvalidTextEncoding,
    InvalidChapterFormat,
    DuplicateChapter,
    PersonNotFound,
    DuplicatePerson,
    DuplicateAlias,
    InvalidAliasTarget,
    InvalidBinaryHeader,
    UnsupportedFileVersion,
    CorruptedProjectFile,
    GraphValidationFailed,
    SaveFailed,
```

```
LoadFailed  
};
```

22.2 错误传播

领域层返回错误信息或抛出领域异常。

应用层将异常转换为 `Result`。

界面层负责：

- 弹出错误提示。
- 在日志区记录错误。
- 恢复按钮状态。
- 保持当前项目不变。

23. 日志设计

日志可保存为文本文件，因为日志不属于项目业务保存数据。

建议日志级别：

- Debug
- Information
- Warning
- Error

记录内容包括：

- 时间。
- 操作类型。
- 文件路径。
- 章节编号。
- 操作结果。
- 错误代码。
- 错误描述。

不得记录完整小说正文，避免日志文件过大。

24. 目录结构

```
NovelRelationAnalyzer/  
├─ CMakeLists.txt  
├─ README.md  
├─ config/  
│   ├── app.ini  
│   ├── persons.txt  
│   └─ aliases.txt  
├─ resources/  
│   ├── icons/  
│   └─ translations/  
├─ src/  
│   ├── main.cpp  
│   ├── domain/  
│   │   ├── model/  
│   │   │   ├── PersonVertex.h  
│   │   │   ├── EdgeNode.h  
│   │   │   ├── ChapterRecord.h  
│   │   │   └─ ProjectMetadata.h  
│   │   ├── graph/  
│   │   │   ├── AdjacencyMultilistGraph.h  
│   │   │   └─ AdjacencyMultilistGraph.cpp  
│   │   ├── chapter/  
│   │   │   ├── ChapterCollection.h  
│   │   │   └─ ChapterCollection.cpp  
│   │   ├── alias/  
│   │   │   ├── AliasDictionary.h  
│   │   │   └─ AliasDictionary.cpp  
│   │   ├── statistics/  
│   │   │   ├── GraphStatisticsBuilder.h  
│   │   │   └─ GraphStatisticsBuilder.cpp  
│   │   ├── traversal/  
│   │   │   ├── GraphTraversalService.h  
│   │   │   ├── GraphTraversalService.cpp  
│   │   │   ├── ChildListTree.h  
│   │   │   └─ ChildListTree.cpp  
│   │   └─ validation/  
│   │       ├── ProjectValidator.h  
│   │       └─ ProjectValidator.cpp  
│   ├── application/  
│   │   ├── ProjectApplicationService.h  
│   │   ├── ProjectApplicationService.cpp  
│   │   ├── dto/  
│   │   └─ command/
```

```

| |   └─ query/
| |   └─ infrastructure/
| |       └─ text/
| |           └─ Utf8TextFileLoader.h
| |           └─ ChapterTextParser.h
| |               └─ DictionaryNameExtractor.h
| |           └─ persistence/
| |               └─ IProjectSerializer.h
| |               └─ BinaryProjectSerializer.h
| |               └─ BinaryProjectSerializer.cpp
| |               └─ BinaryReader.h
| |                   └─ BinaryWriter.h
| |       └─ logging/
| |           └─ ILogger.h
| |               └─ FileLogger.cpp
|   └─ presentation/
|       └─ MainWindow.h
|       └─ MainWindow.cpp
|       └─ dialogs/
|       └─ models/
|           └─ graphics/
|               └─ PersonNodeItem.h
|               └─ RelationEdgeItem.h
|                   └─ RelationGraphSceneController.h
└─ tests/
    └─ domain/
    └─ persistence/
    └─ application/
    └─ data/

```

25. CMake 设计

根目录示例：

```

cmake_minimum_required(VERSION 3.21)

project(
    NovelRelationAnalyzer
    VERSION 1.0.0
    LANGUAGES CXX
)

set(CMAKE_CXX_STANDARD 17)

```

```
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)

find_package(
    Qt6 REQUIRED
    COMPONENTS Widgets Test
)

qt_standard_project_setup()

add_subdirectory(src)
add_subdirectory(tests)
```

核心领域模块应尽量减少 Qt 依赖。

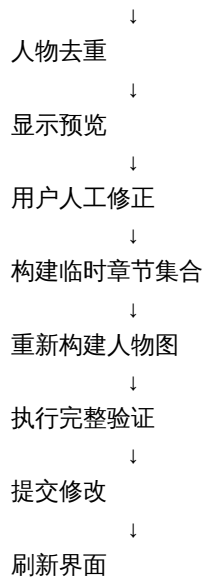
26. 关键业务流程

26.1 新建项目

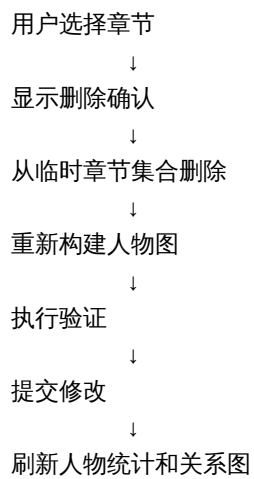
用户选择“新建项目”
↓
选择人物词典和别名词典
↓
读取 UTF-8 文本文件
↓
验证标准人物和别名
↓
创建空项目对象
↓
初始化人物顶点
↓
显示空人物关系图

26.2 导入章节

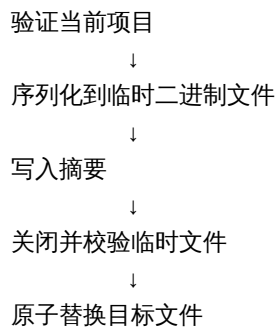
选择章节文本文件
↓
解析章节编号、标题和正文
↓
自动提取人物名称
↓
解析人物别名



26.3 删除章节



26.4 保存项目



↓
更新保存状态

27. 手工边操作与教学演示

系统底层必须实现边插入和边删除。

为了避免手工操作破坏章节统计，可以提供“结构测试模式”。

结构测试模式具有以下特点：

- 从当前人物图创建临时副本。
- 在临时副本中手工插入或删除边。
- 实时显示邻接多重表变化。
- 显示每个顶点的边链。
- 支持验证插入和删除结果。
- 退出结构测试模式后丢弃临时副本。
- 不影响正式章节统计。

正式统计模式中的关系边由章节记录自动生成。

28. 数据表与查询界面

28.1 人物表

字段	说明
人物编号	内部 PersonId
标准人物名	统一名称
出现章节数	$C(A)$
直接关联人数	顶点度
最高关联人物	最大 Jaccard 邻居
最高关联度	最大 Jaccard 值

28.2 关系表

字段	说明
人物 A	第一个端点

字段	说明
人物 B	第二个端点
共同章节数	$C(A, B)$
Jaccard	$J(A, B)$
共同章节	章节编号列表

28.3 章节表

字段	说明
章节编号	唯一 chapterKey
标题	章节标题
人物数量	去重后人物数
来源文件	原始文件名
导入时间	导入时间
状态	正常或待检查

29. 测试方案

29.1 单元测试

邻接多重表测试

- 插入第一条边。
- 插入多条共享端点的边。
- 删除边链头结点。
- 删除边链中间结点。
- 删除边链尾结点。
- 删除不存在的边。
- 拒绝重复边。
- 拒绝自环。
- 删除人物时处理相关边。
- 验证边在两个端点边链中各出现一次。

Jaccard 测试

已知：

A 出现 5 章
B 出现 4 章
A、B 共同出现 3 章

期望：

$$J(A, B) = 0.5$$

还应测试：

- 两个人物只共同出现一次。
- 两个人物每次都共同出现。
- 共同章节数为零。
- 分母为零的防御处理。
- 大量章节下的浮点误差。

章节统计测试

章节一：

A B C

章节二：

A B

章节三：

A D

期望：

$C(A)=3$
 $C(B)=2$
 $C(C)=1$
 $C(D)=1$

 $C(A, B)=2$
 $C(A, C)=1$
 $C(B, C)=1$
 $C(A, D)=1$

DFS/BFS 测试

- 单顶点图。
- 链式图。
- 环形图。
- 星形图。
- 非连通图。
- 含多个可选邻居的图。
- 验证遍历树边数。

29.2 持久化测试

- 保存后重新加载。
- 保存空项目。
- 保存中文人物名。
- 保存长章节正文。
- 读取错误魔数文件。
- 读取不支持的版本。
- 读取被截断文件。
- 读取摘要错误文件。
- 验证加载失败时当前项目不变。
- 验证保存失败时旧文件不被破坏。

29.3 界面测试

- 导入合法章节。
- 导入重复章节。
- 修正人物识别结果。
- 删除章节。
- 搜索人物。
- 设置关联度阈值。
- 点击人物结点。
- 点击关系边。
- 执行 DFS 和 BFS。
- 显示孩子链表遍历树。
- 保存并重新打开项目。

30. 验收标准

系统达到以下条件时可视为完成：

1. 能从文本文件读取标准人物和别名。
2. 能一次导入一个章节文本文件。
3. 能多次导入章节并累计统计。

4. 能对同章人物去重。
 5. 能正确计算人物出现章节数。
 6. 能正确计算共同出现章节数。
 7. 能正确计算 Jaccard 关联度。
 8. 能使用邻接多重表存储人物关系。
 9. 一条无向边只创建一个边结点。
 10. 能插入和删除邻接多重表边。
 11. 能输出全部顶点和全部边。
 12. 能执行 DFS。
 13. 能执行 BFS。
 14. 能生成孩子链表遍历树。
 15. 能图形化显示人物关系。
 16. 能图形化显示遍历树。
 17. 能修改和删除章节。
 18. 每次修改后能够验证数据一致性。
 19. 能将项目保存为二进制文件。
 20. 能从二进制文件恢复完整项目。
 21. 不存在可修改的业务全局变量。
 22. 不使用固定长度数组保存人物、边和章节。
 23. 各模块职责清晰，界面层不直接操作图内部结构。
 24. 加载失败或导入失败时不会破坏现有项目。
-

31. 开发阶段划分

第一阶段：领域数据结构

完成：

- 人物顶点。
- 边结点。
- 邻接多重表。
- 边插入。
- 边删除。
- 图结构验证。

第二阶段：章节与统计

完成：

- 章节记录。
- 人物词典。
- 别名解析。
- 章节人物去重。
- 人物计数。
- 共现计数。

- Jaccard 计算。
- 图重建。

第三阶段：图算法

完成：

- DFS。
- BFS。
- 非连通图遍历。
- 孩子链表。
- 遍历树输出。

第四阶段：文件系统

完成：

- UTF-8 文本读取。
- 章节文本解析。
- 二进制保存。
- 二进制加载。
- 文件摘要。
- 原子保存。

第五阶段：图形界面

完成：

- 主窗口。
- 章节导入界面。
- 人物表。
- 关系表。
- 人物关系图。
- 详情面板。
- 遍历控制。
- 遍历树显示。

第六阶段：测试与完善

完成：

- 单元测试。
- 集成测试。
- 文件损坏测试。
- 图结构验证。
- 性能测试。
- 用户操作流程测试。

32. 编码规范

32.1 命名规范

- 类名使用 `PascalCase`。
- 函数名和变量名使用 `camelCase`。
- 私有成员变量以 `_` 结尾。
- 常量使用 `constexpr`。
- 接口类以 `I` 开头。
- DTO 类型以 `Dto` 结尾。
- 命令类型以 `Command` 结尾。

32.2 资源管理

- 优先使用自动对象。
- 独占所有权使用 `std::unique_ptr`。
- 共享只读资源可使用 `std::shared_ptr<const T>`。
- 非拥有关系可以使用裸指针或引用，并在注释中说明。
- 禁止直接调用业务对象的 `new` 和 `delete`。
- 禁止跨模块传递需要手工释放的内存。

32.3 头文件约束

- 头文件使用前置声明减少依赖。
- 避免在头文件中包含大型 Qt 模块。
- 领域层头文件不包含界面类。
- 不在头文件中使用 `using namespace std`。

32.4 函数约束

- 单个函数尽量不超过 50 行。
- 一个函数只完成一个明确任务。
- 参数较多时使用命令对象。
- 只读参数优先使用 `const` 引用。
- 对可能失败的操作返回 `Result`。
- 不在底层函数中弹窗。

33. 可扩展方向

系统完成基础要求后，可以继续增加：

- 基于 TF-IDF 的人物关系指标。
- 按段落或场景统计人物共现。
- 区分直接对话关系和同章共现关系。

- 人物中心度分析。
- 连通分量分析。
- 最短关系路径查询。
- 人物社群划分。
- 关系变化趋势。
- 按章节范围重新计算。
- 导出人物关系图片。
- 导出 CSV 分析结果。
- 支持多个小说项目。
- 接入更复杂的人名识别模型。

导出的 CSV 或图片属于分析结果，不作为项目恢复文件。项目完整恢复仍以二进制文件为准。

34. 总结

本系统将小说人物共现统计、无向加权图、邻接多重表、DFS、BFS 和孩子链表结合起来。

系统以章节记录作为原始业务数据，以人物图作为派生统计结构。通过分层架构和接口隔离，界面、文件读写、名称提取、统计计算和图算法可以独立修改。

人物、章节和关系数据全部由项目对象管理，不依赖业务全局变量。人物集合、边集合和章节集合使用标准动态容器，不使用固定长度数组。初始词典和小说章节可以从 UTF-8 文本文件读取，程序项目数据采用带版本、区段和摘要校验的二进制格式保存。

该设计能够满足数据结构课程设计要求，也具备较好的可维护性、可测试性和后续扩展能力。